MINI PROJECT 16

Integrasi End-to-End dan Acceptance Test Proyek Final

Durasi
untuk reference acceptance dan live demo bila perangkat serta environment Bab 1–15 sudah siap. Setup OS, driver audio, soldering, dan instalasi toolchain tidak dihitung sebagai waktu Mini Project.

Tujuan: menjalankan artefak beku MP15 sebagai satu sistem, memverifikasi konsistensi terhadap MP13, melakukan live command demo pada Raspberry Pi/PC bila mikrofon tersedia, menghasilkan acceptance report, dan menutup status serial proyek. Untuk ESP32, paket C/header dari MP15–MP16 digunakan sebagai baseline porting dengan acceptance criteria yang sama.

Langkah 1 - Periksa handoff MP15

Pastikan project_status.json berisi ready_for_mp16 dan deployment/mp15_manifest.json memilih classical_svm_int8. Jangan mengubah model, selected feature, split, atau class order.

Langkah 2 - Siapkan environment

Jalankan: pip install numpy scipy matplotlib sounddevice. Pada Raspberry Pi, pastikan mikrofon terlihat oleh ALSA dan Python dijalankan dari root folder running project.

Langkah 3 - Jalankan replay acceptance

Simpan kode sebagai mini_project_16.py. Biarkan LIVE_MIC=False dan jalankan python mini_project_16.py. Periksa agreement dengan prediksi MP13, latency, overflow, dan file acceptance metrics.

Langkah 4 - Jalankan live microphone demo

Pada Raspberry Pi/PC, ubah LIVE_MIC=True lalu jalankan ulang. Ucapkan nyala, mati, naik, turun saat diminta. Catat predicted class dan pipeline latency. Jika salah satu command gagal, simpan sebagai error analysis; jangan retune model menggunakan frozen test.

Langkah 5 - Hubungkan aksi

Map output ke LED/relay/GPIO atau minimal console action. Gunakan mapping nyala=ON, mati=OFF, naik=level+, turun=level-. Pastikan batas minimum/maksimum dan fail-safe.

Langkah 6 - Final acceptance

Buka results/metrics/mp16_acceptance_metrics.json dan deployment/mp16_final_project_manifest.json. Isi benchmark perangkat aktual pada laporan. Bila live system dan seluruh gate lulus, tandai proyek complete dan arsipkan package final.

Checkpoint	PASS bila
Serial gate	ready_for_mp16
Model	sama dengan manifest MP15
Equivalence	agreement terhadap MP13 tidak berubah
Real-time	tidak backlog/overflow pada uji target
Live demo	empat command menghasilkan event terlog
No retuning	post_test_tuning_performed=False
Final package	manifest, metrics, prediction audit, source tersedia

Tabel 16.11. Checklist Mini Project 16.

Kode Python Mini Project 16

Kode berikut adalah reference implementation. Untuk uji tanpa mikrofon gunakan LIVE_MIC=False. Untuk final live demo pada Raspberry Pi/PC, ubah menjadi LIVE_MIC=True. Kode membaca semua parameter dari artefak MP1–MP15 dan tidak membuat model baru.

```
from pathlib import Path
import csv, json, time, hashlib
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import lfilter
from scipy.fft import dct
```

```
ROOT = Path('.')
LIVE_MIC = False     # True pada Raspberry Pi/PC dengan mikrofon
LIVE_DURATION_S = 1.2
SEED = 2026
np.random.seed(SEED)


def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, indent=2), encoding='utf-8')


def sha256(path):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for block in iter(lambda: f.read(65536), b''):
            h.update(block)
    return h.hexdigest()

# ---------- 1. Gate serial MP15 -> MP16 ----------
status_path = ROOT/'project_status.json'
status = json.loads(status_path.read_text(encoding='utf-8'))
if status.get('status') != 'ready_for_mp16':
    raise RuntimeError(f"Status harus ready_for_mp16, ditemukan {status.get('status')!r}")

manifest = json.loads((ROOT/'deployment/mp15_manifest.json').read_text(encoding='utf-8'))
dep_cfg = json.loads((ROOT/'config/deployment_config.json').read_text(encoding='utf-8'))
rt_cfg = json.loads((ROOT/'config/realtime_architecture_config.json').read_text(encoding='utf-8'))
proj_cfg = json.loads((ROOT/'config/project_config.json').read_text(encoding='utf-8'))
pre_cfg = json.loads((ROOT/'config/preprocessing_config.json').read_text(encoding='utf-8'))
feat_cfg = json.loads((ROOT/'config/feature_config.json').read_text(encoding='utf-8'))
mp13 = json.loads((ROOT/'results/metrics/mp13_final_test_metrics.json').read_text(encoding='utf-8'))
mp14 = json.loads((ROOT/'results/metrics/mp14_realtime_metrics.json').read_text(encoding='utf-8'))
mp15 = json.loads((ROOT/'results/metrics/mp15_deployment_metrics.json').read_text(encoding='utf-8'))
split = json.loads((ROOT/'splits/mp09_split.json').read_text(encoding='utf-8'))

if manifest['selected_model'] != dep_cfg['selected_deployment_model']:
    raise RuntimeError('Manifest MP15 dan deployment_config tidak konsisten.')
if dep_cfg.get('post_test_tuning_performed'):
    raise RuntimeError('Post-test tuning terdeteksi. Integrasi dihentikan.')
if manifest['selected_model'] != 'classical_svm_int8':
    raise RuntimeError('Script MP16 ini memakai jalur utama SVM INT8 yang dipilih MP15.')

# ---------- 2. Load frozen front-end and INT8 SVM ----------
FS = int(manifest['sample_rate_hz'])
LABELS = list(manifest['classes'])
filter_npz = np.load(ROOT/'features/intermediate/mp06_filter_final.npz', allow_pickle=True)
B = np.asarray(filter_npz['b'], dtype=float)
A = np.asarray(filter_npz['a'], dtype=float)
N = int(pre_cfg['frame_length_samples'])
H = int(pre_cfg['hop_length_samples'])
TARGET_PEAK = float(pre_cfg['normalization']['target_peak'])
PREEMPH = float(pre_cfg['pre_emphasis_alpha'])
N_MELS = int(feat_cfg['mel']['n_mels'])
N_MFCC = int(feat_cfg['mfcc']['n_mfcc'])
FMIN = float(feat_cfg['mel']['fmin_hz'])
FMAX = float(feat_cfg['mel']['fmax_hz'])

pkg = np.load(ROOT/'models/mp15_classical_int8.npz', allow_pickle=True)
SEL = pkg['selected_indices'].astype(int)
MU = pkg['selected_mean'].astype(float)
SIG = pkg['selected_scale'].astype(float)
XQ_SCALE = float(pkg['input_quant_scale'][0])
WQ = pkg['weight_q'].astype(np.int8)
W_SCALE = pkg['weight_scales'].astype(float)
BIAS = pkg['bias'].astype(float)
CLASSES = pkg['classes'].astype(str).tolist()
assert CLASSES == LABELS


def hz_to_mel(f): return 2595*np.log10(1+np.asarray(f)/700.0)
def mel_to_hz(m): return 700*(10**(np.asarray(m)/2595.0)-1)
def melbank(n_fft):
    m=np.linspace(hz_to_mel(FMIN),hz_to_mel(FMAX),N_MELS+2)
    hz=mel_to_hz(m); bins=np.floor((n_fft+1)*hz/FS).astype(int)
    fb=np.zeros((N_MELS,n_fft//2+1))
    for i in range(1,N_MELS+1):
        l,c,r=bins[i-1],bins[i],bins[i+1]; c=max(c,l+1); r=max(r,c+1)
        for k in range(l,min(c,fb.shape[1])): fb[i-1,k]=(k-l)/(c-l)
        for k in range(c,min(r,fb.shape[1])): fb[i-1,k]=(r-k)/(r-c)
    return fb
FB = melbank(N)
FREQ = np.fft.rfftfreq(N,1/FS)


def read_float(path):
    fs,y = wavfile.read(path)
    if fs != FS: raise RuntimeError(f'{path}: sample rate {fs}, seharusnya {FS}')
    if y.ndim>1: y=y.mean(axis=1)
    if np.issubdtype(y.dtype,np.integer): y=y.astype(np.float64)/np.iinfo(y.dtype).max
    else: y=y.astype(np.float64)
    return y


def preprocess(y):
    y=lfilter(B,A,y)
    y=y-np.mean(y)
    y=y*(TARGET_PEAK/(np.max(np.abs(y))+1e-12))
    if len(y)<N: y=np.pad(y,(0,N-len(y)))
    starts=np.arange(0,len(y)-N+1,H)
    raw=np.array([y[s:s+N] for s in starts])
```

```
    rms=np.sqrt(np.mean(raw**2,axis=1)+1e-12)
    thr=float(pre_cfg['vad']['relative_factor'])*np.max(rms)
    active=np.where(rms>=thr)[0]
    if len(active):
        margin=int(pre_cfg['vad'].get('margin_frames',2))
        i0=max(0,int(active[0])-margin); i1=min(len(raw)-1,int(active[-1])+margin)
        y=y[starts[i0]:min(len(y),starts[i1]+N)]
    if len(y)<N: y=np.pad(y,(0,N-len(y)))
    starts=np.arange(0,len(y)-N+1,H)
    fr=np.array([y[s:s+N] for s in starts])
    fr=np.concatenate([fr[:,:1],fr[:,1:]-PREEMPH*fr[:,:-1]],axis=1)
    return fr*np.hamming(N)[None,:]


def pitch_one(frame):
    x=frame-np.mean(frame); ac=np.correlate(x,x,mode='full')[len(x)-1:]
    if ac[0]<=1e-12: return 0.0
    ac=ac/(ac[0]+1e-12)
    lo=max(1,int(FS/feat_cfg['pitch']['max_hz'])); hi=min(len(ac)-1,int(FS/feat_cfg['pitch']['min_hz']))
    if hi<=lo: return 0.0
    lag=int(np.argmax(ac[lo:hi+1])+lo); conf=float(ac[lag])
    if conf<float(feat_cfg['pitch']['confidence_threshold']): return 0.0
    return float(FS/lag)


def extract39(y):
    frames=preprocess(y)
    rms=np.sqrt(np.mean(frames**2,axis=1)+1e-12)
    zcr=np.mean(np.abs(np.diff(np.signbit(frames),axis=1)),axis=1)
    spec=np.abs(np.fft.rfft(frames,axis=1))+1e-12; power=spec**2
    den=np.sum(power,axis=1)+1e-12
    centroid=np.sum(power*FREQ[None,:],axis=1)/den
    bw=np.sqrt(np.sum(power*(FREQ[None,:]-centroid[:,None])**2,axis=1)/den)
    cum=np.cumsum(power,axis=1); total=cum[:,-1]
    roll=np.array([FREQ[min(np.searchsorted(cum[i],.85*total[i]),len(FREQ)-1)] for i in range(len(frames))])
    pitch=np.array([pitch_one(f) for f in frames]); voiced=pitch>0
    mel=np.maximum(power@FB.T,1e-12); logmel=np.log(mel)
    mfcc=dct(logmel,type=2,axis=1,norm='ortho')[:,:N_MFCC]
    vals=[]
    for j in range(N_MFCC): vals += [float(np.mean(mfcc[:,j])),float(np.std(mfcc[:,j]))]
    for a in [rms,zcr,centroid,bw,roll]: vals += [float(np.mean(a)),float(np.std(a))]
    pv=pitch[voiced]
    vals += [float(np.mean(pv)) if len(pv) else 0.0,float(np.std(pv)) if len(pv) else 0.0,float(np.mean(voiced))]
    v=np.asarray(vals,dtype=np.float32)
    if v.shape!=(39,) or not np.isfinite(v).all(): raise RuntimeError('Feature vector MP16 tidak valid.')
    return v


def predict_int8(v39):
    z=(v39[SEL]-MU)/SIG
    zq=np.clip(np.round(z/XQ_SCALE),-127,127).astype(np.int8)
    acc=WQ.astype(np.int32)@zq.astype(np.int32)
    scores=acc.astype(float)*(W_SCALE*XQ_SCALE)+BIAS
    k=int(np.argmax(scores))
    return CLASSES[k], scores

# ---------- 3. Replay acceptance on frozen test WAVs ----------
meta={}
with (ROOT/'data/metadata/mp09_metadata.csv').open(encoding='utf-8') as f:
    for r in csv.DictReader(f): meta[r['sample_id']]=r
mp13_pred={}
with (ROOT/'results/predictions/mp13_test_predictions.csv').open(encoding='utf-8') as f:
    for r in csv.DictReader(f): mp13_pred[r['sample_id']]=r

rows=[]; feature_ms=[]; infer_ms=[]; total_ms=[]
for sid in split['test_ids']:
    wav_path=ROOT/meta[sid]['path']
    y=read_float(wav_path)
    t0=time.perf_counter_ns(); v=extract39(y); t1=time.perf_counter_ns(); pred,scores=predict_int8(v); t2=time.perf_counter_ns()
    fm=(t1-t0)/1e6; im=(t2-t1)/1e6; tm=(t2-t0)/1e6
    expected=meta[sid]['label']; frozen=mp13_pred[sid]['classical_prediction']
    rows.append({'sample_id':sid,'expected':expected,'mp13_classical_pred':frozen,'mp16_int8_pred':pred,'correct':pred==expected,'agreement_mp13':pred==frozen,
                 'feature_ms':fm,'inference_ms':im,'total_ms':tm,'audio_duration_ms':len(y)/FS*1000})
    feature_ms.append(fm); infer_ms.append(im); total_ms.append(tm)

# ---------- 4. Optional live microphone demonstration ----------
live_rows=[]
if LIVE_MIC:
    import sounddevice as sd
    for expected in ['nyala','mati','naik','turun']:
        input(f'Ucapkan "{expected}" lalu tekan ENTER... ')
        y=sd.rec(int(LIVE_DURATION_S*FS),samplerate=FS,channels=1,dtype='float32'); sd.wait(); y=y[:,0].astype(float)
        t0=time.perf_counter_ns(); v=extract39(y); pred,scores=predict_int8(v); elapsed=(time.perf_counter_ns()-t0)/1e6
        live_rows.append({'expected':expected,'predicted':pred,'correct':pred==expected,'pipeline_ms':elapsed})
        print(f'  {expected} -> {pred} ({elapsed:.2f} ms)')

# ---------- 5. Acceptance metrics and final package ----------
replay_accuracy=float(np.mean([r['correct'] for r in rows]))
agreement=float(np.mean([r['agreement_mp13'] for r in rows]))
rtf_p95=float(np.percentile(total_ms,95)/(LIVE_DURATION_S*1000))
stride_ms=float(rt_cfg['inference_stride_ms'])

checks={
    'serial_gate_ready_for_mp16':True,
    'selected_model_matches_manifest':True,
    'post_test_tuning_false':not dep_cfg.get('post_test_tuning_performed',False),
    'mp15_numeric_agreement_test_ge_095':float(mp15['svm_int8']['agreement_test_vs_float'])>=0.95,
    'mp14_overflow_is_0':int(mp14['overflow_count'])==0,
    'replay_agreement_not_worse_than_mp15':agreement+1e-9>=float(mp15['svm_int8']['agreement_test_vs_float']),
    'replay_features_are_finite':True,
    'p95_pipeline_faster_than_audio_duration':rtf_p95<1.0,
```

```
}
# Live microphone is informative, not a hard gate because acoustics and hardware vary.
all_required_pass=all(checks.values())

metrics={
  'selected_model':manifest['selected_model'],
  'target_profile':dep_cfg['primary_target_profile'],
  'n_replay_samples':len(rows),
  'replay_accuracy':replay_accuracy,
  'replay_agreement_with_mp13':agreement,
  'feature_latency_ms':{'mean':float(np.mean(feature_ms)),'p95':float(np.percentile(feature_ms,95))},
  'inference_latency_ms':{'mean':float(np.mean(infer_ms)),'p95':float(np.percentile(infer_ms,95))},
  'pipeline_latency_ms':{'mean':float(np.mean(total_ms)),'p95':float(np.percentile(total_ms,95))},
  'pipeline_rtf_p95':rtf_p95,
  'realtime_stride_ms':stride_ms,
  'mp14_overflow_count':int(mp14['overflow_count']),
  'mp15_payload_bytes':int(mp15['svm_int8']['payload_bytes_estimate']),
  'live_microphone_enabled':LIVE_MIC,
  'live_demo':live_rows,
  'acceptance_checks':checks,
  'required_acceptance_pass':all_required_pass,
  'scientific_note':'Replay test verifies end-to-end implementation consistency; it is not a new independent generalization estimate.'
}

out_pred=ROOT/'results/predictions/mp16_end_to_end_predictions.csv'; out_pred.parent.mkdir(parents=True,exist_ok=True)
with out_pred.open('w',newline='',encoding='utf-8') as f:
  fields=list(rows[0].keys()); w=csv.DictWriter(f,fieldnames=fields); w.writeheader(); w.writerows(rows)
save_json(ROOT/'results/metrics/mp16_acceptance_metrics.json', metrics)

syscfg={
  'project':'four_command_keyword_spotting_embedded',
  'classes':LABELS,
  'sample_rate_hz':FS,
  'input':'mono_PCM16_or_float_equivalent',
  'dma_block_samples':int(manifest['dma_block_samples']),
  'frame_samples':int(manifest['feature_frame_samples']),
  'hop_samples':int(manifest['feature_hop_samples']),
  'inference_stride_frames':int(manifest['inference_stride_frames']),
  'selected_model':manifest['selected_model'],
  'selected_features':manifest['selected_feature_names'],
  'decision_map':{'nyala':'LED/relay ON','mati':'LED/relay OFF','naik':'increase level','turun':'decrease level'},
  'logging_fields':['timestamp','predicted_class','score','pipeline_latency_ms','overflow_count'],
  'live_python_target':'Raspberry Pi / PC Linux',
  'esp32_package':'firmware/mp15_classical_model_int8.h',
  'no_post_test_tuning':True,
}
save_json(ROOT/'config/system_integration_config.json', syscfg)

# MCU constants and simple action mapping.
fw=ROOT/'firmware/mp16_project_config.h'; fw.parent.mkdir(parents=True,exist_ok=True)
fw.write_text(f'''// Generated by Mini Project 16\n#pragma once\n#define MP16_SAMPLE_RATE {FS}\n#define MP16_DMA_BLOCK {manifest['dma_block_samples']}\n#define
MP16_FRAME_SAMPLES {manifest['feature_frame_samples']}\n#define MP16_HOP_SAMPLES {manifest['feature_hop_samples']}\n#define MP16_INFERENCE_STRIDE_FRAMES
{manifest['inference_stride_frames']}\n#define MP16_N_CLASSES {len(LABELS)}\n// Class order: {', '.join(LABELS)}\n''',encoding='utf-8')

final_manifest={
  'project_complete':all_required_pass,
  'selected_model':manifest['selected_model'],
  'primary_runtime':'Raspberry Pi/Python reference or ESP32/C++ port using frozen header',
  'model_package':'models/mp15_classical_int8.npz',
  'model_c_header':'firmware/mp15_classical_model_int8.h',
  'project_c_header':'firmware/mp16_project_config.h',
  'system_config':'config/system_integration_config.json',
  'acceptance_metrics':'results/metrics/mp16_acceptance_metrics.json',
  'prediction_audit':'results/predictions/mp16_end_to_end_predictions.csv',
  'frozen_test_metrics':'results/metrics/mp13_final_test_metrics.json',
  'deployment_metrics':'results/metrics/mp15_deployment_metrics.json',
  'post_test_tuning_performed':False,
}
save_json(ROOT/'deployment/mp16_final_project_manifest.json', final_manifest)

# ---------- 6. Figures ----------
figdir=ROOT/'results/figures'; figdir.mkdir(parents=True,exist_ok=True)
fig,ax=plt.subplots(figsize=(7.2,4.0))
ax.bar(['feature','inference','total'],[np.mean(feature_ms),np.mean(infer_ms),np.mean(total_ms)])
ax.set_ylabel('mean latency (ms)'); ax.set_title('MP16 end-to-end reference latency'); ax.grid(axis='y',alpha=.2)
fig.tight_layout(); fig.savefig(figdir/'mp16_latency_breakdown.png',dpi=160); plt.close(fig)

fig,ax=plt.subplots(figsize=(7.2,4.0))
labels=list(checks.keys()); vals=[1 if checks[k] else 0 for k in labels]
ax.barh(range(len(labels)),vals); ax.set_yticks(range(len(labels))); ax.set_yticklabels([x.replace('_',' ') for x in labels],fontsize=7)
ax.set_xlim(0,1.05); ax.set_xlabel('PASS = 1'); ax.set_title('MP16 acceptance checklist'); ax.grid(axis='x',alpha=.2)
fig.tight_layout(); fig.savefig(figdir/'mp16_acceptance_summary.png',dpi=160); plt.close(fig)

fig,ax=plt.subplots(figsize=(7.4,4.0))
x=np.arange(len(rows)); ax.plot(x,[r['total_ms'] for r in rows],marker='o',label='pipeline')
ax.axhline(LIVE_DURATION_S*1000,ls='--',label='audio duration'); ax.set_xticks(x); ax.set_xticklabels([r['sample_id'] for r in rows],rotation=25,ha='right',fontsize=8)
ax.set_ylabel('ms'); ax.set_title('MP16 replay processing time'); ax.legend(); ax.grid(alpha=.2)
fig.tight_layout(); fig.savefig(figdir/'mp16_command_timeline.png',dpi=160); plt.close(fig)

# ---------- 7. Final serial status ----------
status.update({
  'status':'project_complete',
  'last_completed_mini_project':16,
  'mp16':{
    'system_config':'config/system_integration_config.json',
    'acceptance_metrics':'results/metrics/mp16_acceptance_metrics.json',
    'final_manifest':'deployment/mp16_final_project_manifest.json',
    'prediction_audit':'results/predictions/mp16_end_to_end_predictions.csv',
    'selected_model':manifest['selected_model'],
    'required_acceptance_pass':all_required_pass,
    'post_test_tuning_performed':False
```

```
  }
})
status.setdefault('completed_mini_projects',[])
if 16 not in status['completed_mini_projects']: status['completed_mini_projects'].append(16)
status_path.write_text(json.dumps(status,indent=2),encoding='utf-8')

print('MP16 selesai.')
print('selected model =', manifest['selected_model'])
print('replay accuracy =', replay_accuracy)
print('agreement with MP13 =', agreement)
print('p95 pipeline = %.3f ms, RTF = %.4f' % (metrics['pipeline_latency_ms']['p95'], rtf_p95))
print('acceptance =', 'PASS' if all_required_pass else 'FAIL')
print('status = project_complete')
```

Artefak yang dihasilkan

- config/system_integration_config.json
- firmware/mp16_project_config.h
- results/metrics/mp16_acceptance_metrics.json
- results/predictions/mp16_end_to_end_predictions.csv
- deployment/mp16_final_project_manifest.json
- results/figures/mp16_latency_breakdown.png
- results/figures/mp16_acceptance_summary.png
- results/figures/mp16_command_timeline.png
- project_status.json

Makna status akhir

Dalam reference/replay mode, kode memvalidasi integrasi perangkat lunak dan konsistensi model. Untuk klaim bahwa deployment fisik selesai, mahasiswa harus menyertakan hasil live microphone dan benchmark perangkat yang benar-benar digunakan. Jangan mengganti angka host sebagai angka ESP32/Raspberry Pi.

Referensi

1. Warden, P., & Situnayake, D. (2019). TinyML. O'Reilly Media.

2. Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2020). Efficient Processing of Deep Neural Networks. Morgan & Claypool.

3. Lyons, R. G. (2011). Understanding Digital Signal Processing (3rd ed.). Pearson.

4. Oppenheim, A. V., & Schafer, R. W. (2010). Discrete-Time Signal Processing (3rd ed.). Pearson.

5. Rabiner, L. R., & Schafer, R. W. (2010). Theory and Applications of Digital Speech Processing. Pearson.

6. Espressif Systems. ESP-IDF Programming Guide and ESP-DSP documentation.

7. Raspberry Pi Ltd. Raspberry Pi documentation: audio, GPIO, and Linux system integration.

8. ARM. CMSIS-DSP and CMSIS-NN documentation.